

ZAAWANSOWANE PROGRAMOWANIE OBIEKTOWE

Sprawozdanie z Laboratorium nr 5

Aplikacja webowa konsumująca REST API (Spring MVC + Thymeleaf)

Laboratorium nr	5
Tytuł	Aplikacja webowa konsumująca REST API (Spring MVC + Thymeleaf)
Student	Szymon Banaszek
Prowadzacy	dr inż. Damian Szczegielniak
Data	2026-06-09

1. Cel cwiczenia

Celem laboratorium było zbudowanie aplikacji webowej opartej na frameworku Spring MVC, która pełni rolę klienta dla REST API zaimplementowanego w poprzednim laboratorium (LAB04). Aplikacja wyświetla i umożliwia zarządzanie danymi (projekty, studenci) za pomocą interfejsu HTML generowanego po stronie serwera z użyciem silnika szablonów Thymeleaf.

Główne zadania:

- Konfiguracja projektu Spring Boot z Thymeleaf i Spring Web.
- Użycie RestClient do komunikacji z REST API przez Basic Auth.
- Wyświetlanie list projektów i studentów w tabelach HTML.
- Implementacja pełnego zarządzania studentami: dodawanie, edycja i usuwanie (zadanie obowiązkowe z punktu 13 instrukcji).

2. Środowisko i technologie

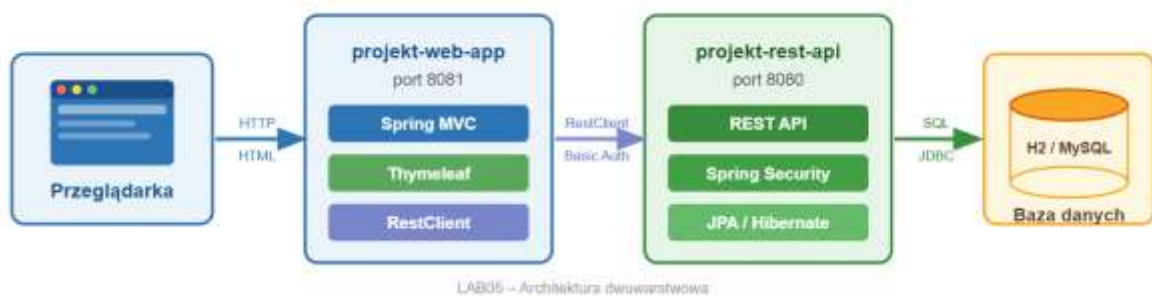
Technologia	Wersja / opis
Java (Temurin)	21 LTS
Spring Boot	3.4.3
Spring Web MVC	wbudowany w Spring Boot
Thymeleaf	wbudowany w Spring Boot
Spring Validation	wbudowany w Spring Boot
Spring Data Commons	PageImpl, Pageable
Lombok	adnotacje @Data, @Builder, @AllArgsConstructor itp.
Jackson Datatype JSR310	serializacja dat Java 8+ (LocalDate, LocalDateTime)
Gradle	8.x (Wrapper)
IDE	IntelliJ IDEA
Port aplikacji webowej	8081
Port REST API (LAB04)	8080

3. Architektura systemu

System składa się z dwóch odrębnych procesów działających równocześnie:

- projekt-rest-api (LAB04) - serwis REST nasłuchujący na porcie 8080, połączony z bazą danych.
- projekt-web-app (LAB05) - aplikacja MVC nasłuchująca na porcie 8081, komunikująca się z LAB04 przez HTTP.

Przepływa danych: przeglądarka wysyła zadanie HTTP do kontrolera Spring MVC, który wywołuje serwis korzystający z RestClient, a ten wysyła zapytanie do REST API na porcie 8080. Odpowiedź JSON jest deserializowana do klas modelu i przekazywana do szablonu Thymeleaf, który generuje HTML zwracany przeglądarce.



4. Generowanie projektu

Projekt został wygenerowany za pomocą Spring Initializr (<https://start.spring.io>) z zależności: Spring Web, Thymeleaf, Validation, Lombok oraz Spring Dev Tools. Następnie ręcznie dodano spring-data-commons oraz jackson-datatype-jsr310 do pliku build.gradle.



5. Konfiguracja projektu

5.1 build.gradle

Plik build.gradle definiuje zależności i wersje Java przez toolchain (Java 21). Kluczowe zależności:

```
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
implementation 'org.springframework.boot:spring-boot-starter-web'
implementation 'org.springframework.data:spring-data-commons'
implementation 'com.fasterxml.jackson.datatype:jackson-datatype-jsr310'
```

5.2 application.properties

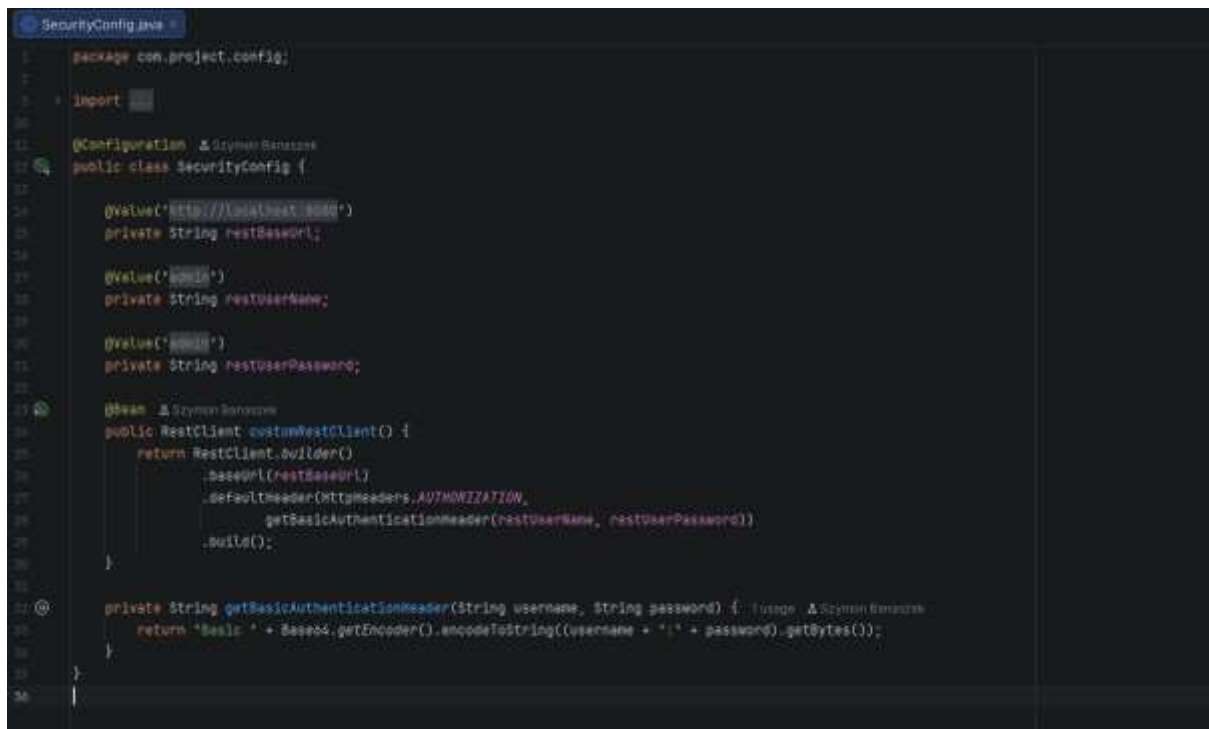
Konfiguracja portu serwera i danych dostępowych do REST API:

```
server.port=8081
rest.base.url=http://localhost:8080
rest.user.name=admin
rest.user.password=admin
```

5.3 SecurityConfig - konfiguracja RestClient

Klasa SecurityConfig tworzy bean RestClient skonfigurowany z bazowym adresem URL i nagłówkiem Basic Auth. Hasło i login są wstrzykiwane z application.properties przez @Value:

```
@Bean
public RestClient customRestClient() {
    return RestClient.builder().baseUrl(restBaseUrl)
        .defaultHeader(HttpHeaders.AUTHORIZATION,
            "Basic " + Base64.getEncoder()
                .encodeToString((user+":"+pass).getBytes()))
        .build();
}
```



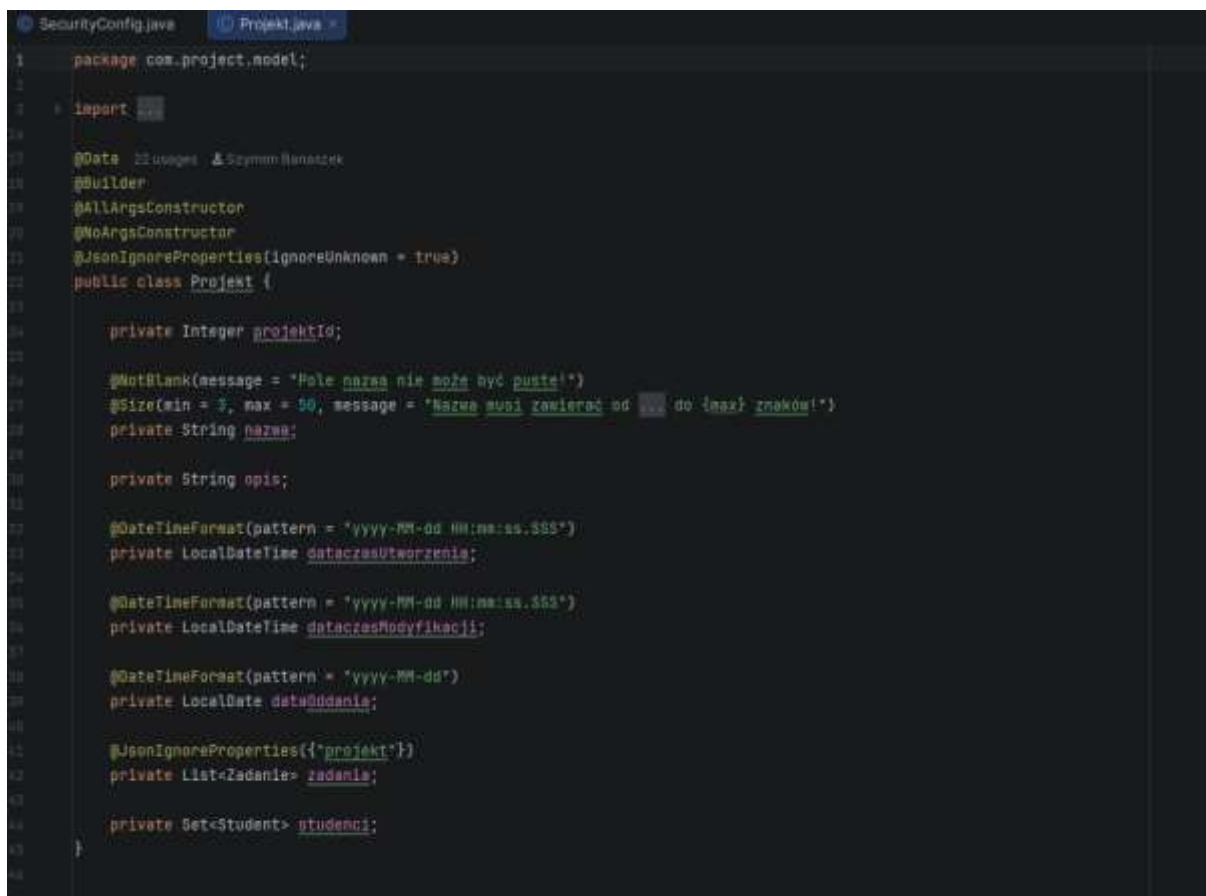
6. Klasy modelu

Klasy modelu nie używają adnotacji JPA. Są to proste POJO ze:

- `@JsonIgnoreProperties(ignoreUnknown = true)` - ignoruje nieznane pola JSON z REST API.
- `@DateTimeFormat(pattern = ...)` - formatowanie dat dla Thymeleaf i wiązania formularzy.
- Adnotacjami walidacyjnymi: `@NotBlank`, `@Size`, `@NotNull`, `@Email`.
- Adnotacjami Lombok: `@Data`, `@Builder`, `@AllArgsConstructor`, `@NoArgsConstructor`.

6.1 Projekt

Pola: `projectId` (Integer), `nazwa` (`@NotBlank`, `@Size(3-50)`), `opis` (String), `dataczasUtworzenia` i `dataczasModyfikacji` (`LocalDateTime`, `@DateTimeFormat yyyy-MM-dd HH:mm:ss.SSS`), `dataOddania` (`LocalDate`, `@DateTimeFormat yyyy-MM-dd`), `zadania` (`List<Zadanie>`), `studenci` (`Set<Student>`).



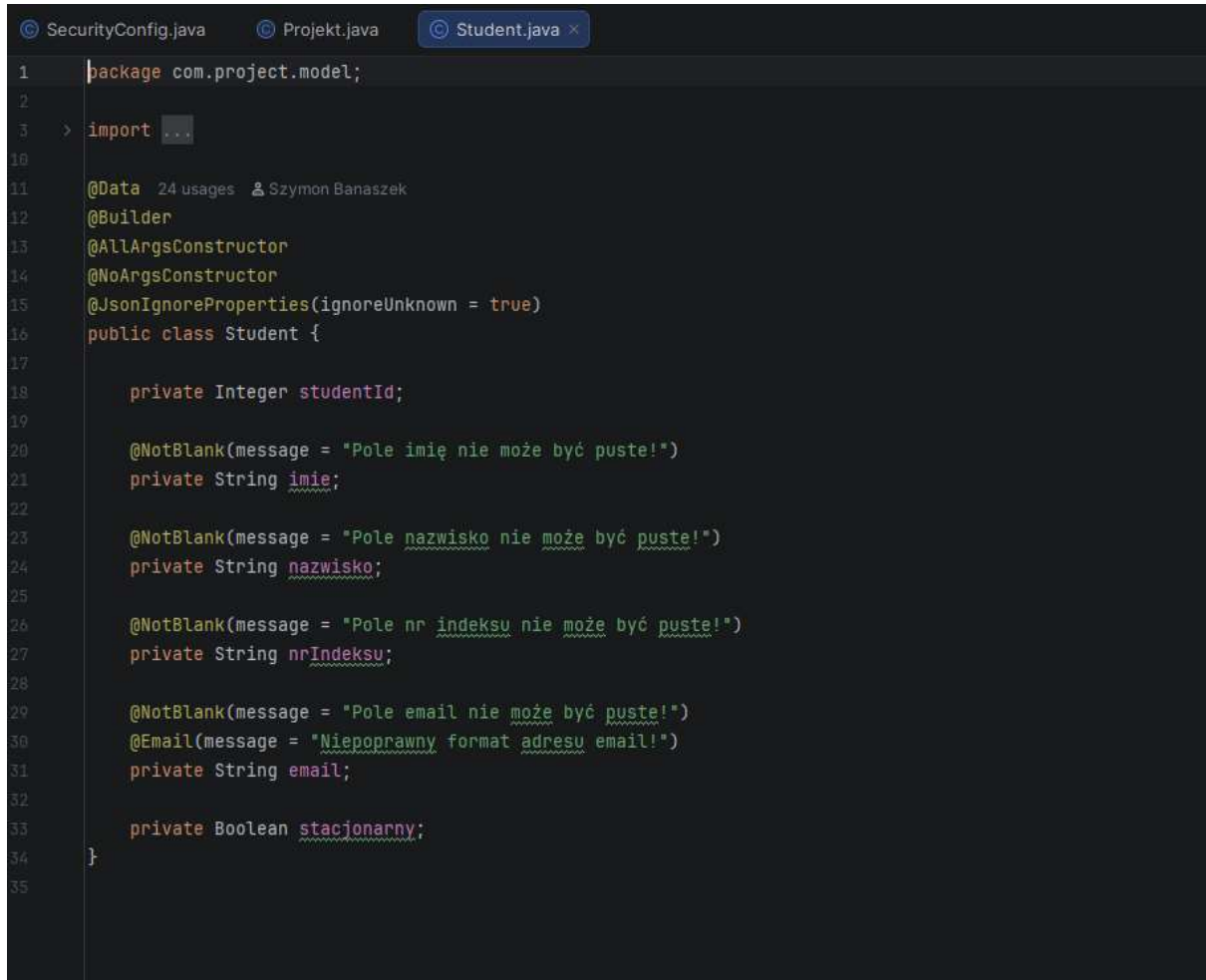
```

1  package com.project.model;
2
3  import ...
4
5  @Data // usages & Szymon Banaszek
6  @Builder
7  @AllArgsConstructor
8  @NoArgsConstructor
9  @JsonIgnoreProperties(ignoreUnknown = true)
10 public class Projekt {
11
12     private Integer projectId;
13
14     @NotBlank(message = "Pole nazwa nie może być puste!")
15     @Size(min = 3, max = 50, message = "Nazwa musi zawierać od ... do (max) znaków!")
16     private String nazwa;
17
18     private String opis;
19
20     @DateTimeFormat(pattern = "yyyy-MM-dd HH:mm:ss.SSS")
21     private LocalDateTime dataczasUtworzenia;
22
23     @DateTimeFormat(pattern = "yyyy-MM-dd HH:mm:ss.SSS")
24     private LocalDateTime dataczasModyfikacji;
25
26     @DateTimeFormat(pattern = "yyyy-MM-dd")
27     private LocalDate dataOddania;
28
29     @JsonIgnoreProperties({"projectId"})
30     private List<Zadanie> zadania;
31
32     private Set<Student> studenci;
33 }

```

6.2 Student

Pola: `studentId` (Integer), `imie` (`@NotBlank`), `nazwisko` (`@NotBlank`), `nrIndeksu` (`@NotBlank`), `email` (`@NotBlank`, `@Email`), `stacjonarny` (Boolean).



```
1 package com.project.model;
2
3 > import ...
4
5
6
7
8
9
10
11 @Data 24 usages Szymon Banaszek
12 @Builder
13 @AllArgsConstructor
14 @NoArgsConstructor
15 @JsonIgnoreProperties(ignoreUnknown = true)
16 public class Student {
17
18     private Integer studentId;
19
20     @NotBlank(message = "Pole imię nie może być puste!")
21     private String imię;
22
23     @NotBlank(message = "Pole nazwisko nie może być puste!")
24     private String nazwisko;
25
26     @NotBlank(message = "Pole nr indeksu nie może być puste!")
27     private String nrIndeksu;
28
29     @NotBlank(message = "Pole email nie może być puste!")
30     @Email(message = "Niepoprawny format adresu email!")
31     private String email;
32
33     private Boolean stacjonarny;
34 }
35
```

6.3 Zadanie

Pola: zadanield (Integer), nazwa (@NotBlank, @Size(3-50)), kolejnosc (Integer), opis (@NotNull, @Size(max=1000)), dataczasDodania (LocalDateTime, @DateTimeFormat), projekt (@JsonIgnoreProperties({"projekt"}) - zapobiega nieskończonej rekurencji JSON).

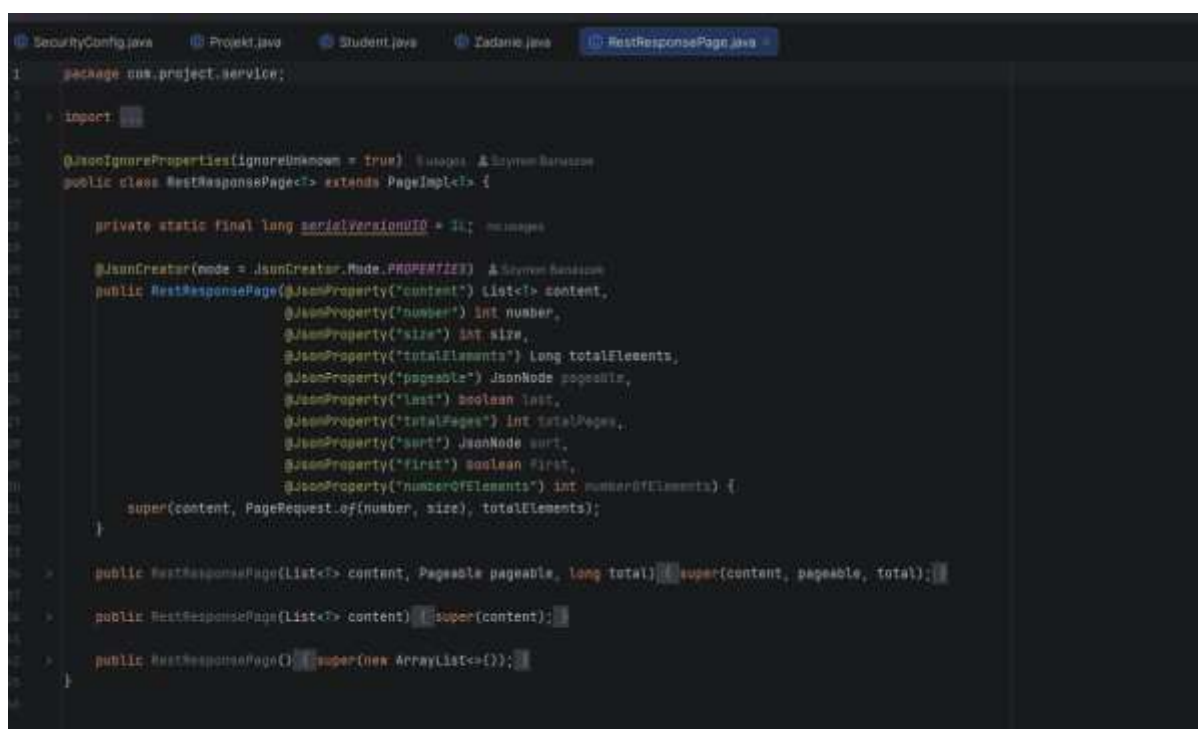
```
SecurityConfig.java Projekt.java Student.java Zadanie.java
1 package com.project.model;
2
3 import java.util.*;
4
5 @Data
6 @Builder
7 @AllArgsConstructor
8 @NoArgsConstructor
9 @JsonIgnoreProperties(ignoreUnknown = true)
10 public class Zadanie {
11
12     private Integer zadanieId;
13
14     @NotBlank(message = "Pole nazwa nie może być puste!")
15     @Size(min = 3, max = 50, message = "Nazwa musi zawierać od {min} do {max} znaków!")
16     private String nazwa;
17
18     private Integer kolejnosc;
19
20     @NotNull(message = "Pole opis nie może być puste!")
21     @Size(max = 1000, message = "Pole opis może zawierać maksymalnie {max} znaków!")
22     private String opis;
23
24     @DateTimeFormat(pattern = "yyyy-MM-dd HH:mm:ss.SSS")
25     private LocalDateTime dataCzasDodania;
26
27     @JsonIgnoreProperties({"projekt"})
28     private Projekt projekt;
29 }
```

7. Klasy pomocnicze

7.1 RestResponsePage<T>

Klasa `RestResponsePage<T>` rozszerza `PageImpl<T>` i umożliwia deserializację stronicowanych odpowiedzi JSON z REST API. Konstruktor z `@JsonCreator` mapuje pola JSON (`content`, `number`, `size`, `totalElements`) na parametry `PageImpl`:

```
@JsonIgnoreProperties(ignoreUnknown = true)
public class RestResponsePage<T> extends PageImpl<T> {
    @JsonCreator
    public RestResponsePage(
        @JsonProperty("content") List<T> content,
        @JsonProperty("number") int number,
        @JsonProperty("size") int size,
        @JsonProperty("totalElements") long totalElements) {
        super(content, PageRequest.of(number, size), totalElements);
    }
}
```



7.2 ServiceUtil

Klasa narzędzi `ServiceUtil` zawiera metody statyczne do budowania URI z parametrami `Pageable` (`page`, `size`, `sort`) oraz do wykonywania zapytania GET z oczekiwaniem stronicowanej odpowiedzi `RestResponsePage<T>`.

```
SecurityConfig.java Projekt.java Student.java Zadanie.java RestResponsePage.java ServiceUtil.java

public class ServiceUtil {
    // ...

    public static <T> RestResponsePage<T> getPage(Uri uri, RestClient restClient,
        ParameterizedTypeReference<RestResponsePage<T>> responseType) {
        return restClient.get()
            .uri(uri)
            .retrieve()
            .body(responseType);
    }

    // ...

    public static Uri getUri(String resourcePath, Pageable pageable) {
        return getUriComponent(resourcePath)
            .queryParams("page", pageable.getPageNumber())
            .queryParams("size", pageable.getPageSize())
            .queryParams("sort", ServiceUtil.getSortParams(pageable.getSort()))
            .build()
            .toUri();
    }

    // ...

    public static UriComponentsBuilder getUriComponent(String resourcePath, Pageable pageable) {
        return getUriComponent(resourcePath)
            .queryParams("page", pageable.getPageNumber())
            .queryParams("size", pageable.getPageSize())
            .queryParams("sort", ServiceUtil.getSortParams(pageable.getSort()));
    }

    // ...

    public static UriComponentsBuilder getUriComponent(String resourcePath) {
        return UriComponentsBuilder.fromUriString(resourcePath);
    }

    // ...

    public static String getSortParams(Sort sort) {
        StringBuilder builder = new StringBuilder();
        if (sort != null) {
            String sep = "";
            for (Sort.Order order : sort) {
                builder.append(sep)
                    .append(order.getProperty())
                    .append(",")
                    .append(order.getDirection());
                sep = "And";
            }
        }
        return builder.toString();
    }
}
```

7.3 HttpException

Klasa `HttpException` rozszerza `RuntimeException` i służy do opakowywania błędów HTTP z `RestClient`. Umożliwia propagowanie kodu statusu HTTP i nagłówków odpowiedzi do warstwy kontrolera.

```
SecurityConfig.java Projekt.java Student.java Zadanie.java RestResponsePage.java ServiceUtil.java HttpException.java

package coe.project.exception;

import java.io.IOException;

public class HttpException extends RuntimeException {
    private static final long serialVersionUID = 1L;

    public HttpException(HttpStatus httpStatus, HttpHeaders httpHeaders) {
        super(String.format("Error: status: %s, headers: %s", httpStatus, httpHeaders));
    }

    public HttpException(String errorMessage) {
        super(errorMessage);
    }

    public HttpException(String errorMessage, Throwable err) {
        super(errorMessage, err);
    }
}
```

8. Warstwa serwisow (RestClient)

Klasy serwisowe implementują odpowiednie interfejsy i komunikują się z REST API przez wstrzyknięty bean RestClient. Każda klasa obsługuje operacje CRUD na swoim zasobie.

8.1 ProjektServiceImpl

Operacje: getProjekty (GET /api/projekty z paginacją), getProjekt (GET /api/projekty/{id}), setProjekt (PUT jeśli projectId != null, inaczej POST + GET lokalizacji z Location header), deleteProjekt (DELETE /api/projekty/{id}).

```

package com.project.service;

import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClientException;
import org.springframework.web.client.RestClient;

@Service
public class ProjektServiceImpl implements ProjektService {

    private static final Logger logger = LoggerFactory.getLogger(ProjektServiceImpl.class);

    private final RestClient restClient;

    public ProjektServiceImpl(RestClient restClient) {
        this.restClient = restClient;
    }

    private String getResourcePath() {
        return "/api/projekty";
    }

    private String getResourcePath(Integer id) {
        return String.format("%s/%d", getResourcePath(), id);
    }

    @Override
    public Optional<Projekt> getProjekt(Integer projectId) {
        String resourcePath = getResourcePath(projectId);
        logger.info("REQUEST -> GET {}", resourcePath);
        Projekt projekt = restClient
            .get()
            .uri(resourcePath)
            .accept(MediaType.APPLICATION_JSON)
            .retrieve()
            .getBody();
        logger.info("RESPONSE -> GET {}", resourcePath);
        return Optional.ofNullable(projekt);
    }

    @Override
    public Projekt setProjekt(Projekt projekt) {
        if (projekt.getProjectId() != null) {
            String resourcePath = getResourcePath(projekt.getProjectId());
            logger.info("REQUEST -> PUT {}", resourcePath);
            restClient
                .put()
                .uri(resourcePath)
                .body(projekt)
                .accept(MediaType.APPLICATION_JSON)
                .retrieve()
                .getBody();
            logger.info("RESPONSE -> PUT {}", resourcePath);
        } else {
            String resourcePath = getResourcePath();
            logger.info("REQUEST -> POST {}", resourcePath);
            restClient
                .post()
                .uri(resourcePath)
                .body(projekt)
                .accept(MediaType.APPLICATION_JSON)
                .retrieve()
                .getBody();
            logger.info("RESPONSE -> POST {}", resourcePath);
        }
        return projekt;
    }

    @Override
    public void deleteProjekt(Integer projectId) {
        String resourcePath = getResourcePath(projectId);
        logger.info("REQUEST -> DELETE {}", resourcePath);
        restClient
            .delete()
            .uri(resourcePath)
            .accept(MediaType.APPLICATION_JSON)
            .retrieve()
            .getBody();
        logger.info("RESPONSE -> DELETE {}", resourcePath);
    }
}

```

8.2 StudentServiceImpl

Analogiczna implementacja dla zasobu /api/studenci. Dodatkowe metody: searchByNazwisko i searchByNrIndeksu (zapytania GET z parametrami wyszukiwania).

```
@Service
public class StudentServiceImpl implements StudentService {

    private static final Logger logger = LoggerFactory.getLogger(StudentServiceImpl.class);

    private final RestClient restClient;

    public StudentServiceImpl(RestClient restClient) { this.restClient = restClient; }

    private String getResourcePath() { return "/api/studenci"; }

    private String getResourcePath(Integer id) { return String.format("%s/%d", getResourcePath(), id); }

    @Override
    public Optional<Student> getStudent(Integer studentId) {
        String resourcePath = getResourcePath(studentId);
        logger.info("REQUEST -> GET {}", resourcePath);
        Student student = restClient
            .get()
            .uri(resourcePath)
            .retrieve()
            .onStatus(HttpStatusCode::isError, (HttpRequest req, HttpResponse res) -> {
                throw new HttpException(res.getStatusCode(), res.getHeaders());
            })
            .body(Student.class);
        return Optional.ofNullable(student);
    }

    @Override
    public Student getStudent(Student student) {
        if (student.getId() != null) {
            String resourcePath = getResourcePath(student.getId());
            logger.info("REQUEST -> PUT {}", resourcePath);
            restClient
                .put()
                .uri(resourcePath)
                .accept(MediaType.APPLICATION_JSON)
                .body(student)
                .retrieve()
                .onStatus(HttpStatusCode::isError, (HttpRequest req, HttpResponse res) -> {
                    throw new HttpException(res.getStatusCode(), res.getHeaders());
                })
                .toEntity(student);
            return student;
        } else {
            String resourcePath = getResourcePath();
        }
    }
}
```

8.3 ZadanieServiceImpl

Implementacja dla zasobu /api/zadania. Zawiera metode getZadaniaProjektu pobierajaca zadania dla danego projektu przez zapytanie GET z parametrem projektId.

```
public class ZadanieServiceImpl implements ZadanieService {

    @Override @Transactional
    public Optional<Zadanie> getZadanie(Integer zadanieId) {
        String resourcePath = getResourcePath(zadanieId);
        logger.info("REQUEST -> GET {}", resourcePath);
        Zadanie zadanie = restTemplate
            .get()
            .headers(HttpHeaders.ACCEPT)
            .uri(resourcePath)
            .contentType(MediaType.APPLICATION_JSON)
            .retrieve()
            .getBody();
        Optional.ofNullable(zadanie);
    }

    @Override @Transactional
    public Zadanie putZadanie(Zadanie zadanie) {
        if (zadanie.getZadanieId() != null) {
            String resourcePath = getResourcePath(zadanie.getZadanieId());
            logger.info("REQUEST -> PUT {}", resourcePath);
            restTemplate
                .put()
                .headers(HttpHeaders.ACCEPT)
                .uri(resourcePath)
                .contentType(MediaType.APPLICATION_JSON)
                .body(zadanie)
                .retrieve()
                .getBody();
            Optional.ofNullable(zadanie);
        } else {
            String resourcePath = getResourcePath();
            logger.info("REQUEST -> POST {}", resourcePath);
            ResponseEntity<Zadanie> response = restTemplate
                .post()
                .headers(HttpHeaders.ACCEPT)
                .uri(resourcePath)
                .contentType(MediaType.APPLICATION_JSON)
                .body(zadanie)
                .retrieve()
                .getBody();
            Optional.ofNullable(response.getBody());
        }
    }
}
```

9. Kontrolery MVC

Kontrolery sa oznaczone `@Controller` i zwracaja nazwy szablonow Thymeleaf lub przekierowania `redirect:/...`. Walidacja danych formularza odbywa sie przez `@Valid` i `BindingResult` - bledy sa zwracane do formularza bez przekierowania.

9.1 ProjectController

Endpointy `/projektList` i `/projektEdit`:

- GET `/projektList` - pobiera liste projektow i przekazuje do szablonu jako atrybut modelu.
- GET `/projektEdit?projectId=X` - laduje projekt o podanym ID (lub tworzy nowy pusty obiekt).
- POST `/projektEdit` - zapisuje projekt, wraca do formularza jesli sa bledy walidacji.
- POST `/projektEdit?cancel` - anuluje i przekierowuje na `/projektList`.
- POST `/projektEdit?delete` - usuwa projekt i przekierowuje na `/projektList`.

```

1 package com.project.controller;
2
3 import
4
5
6
7 @Controller @RequestMapping("/projekt")
8 public class ProjectController {
9
10     private ProjectService projektService;
11
12     public ProjectController(ProjectService projektService) { this.projektService = projektService; }
13
14     @GetMapping("/projektList")
15     public String projektList(Model model, Pageable pageable) {
16         model.addAttribute("projektList", projektService.getProjekty(pageable).getContent());
17         return "projektList";
18     }
19
20     @GetMapping("/projektEdit")
21     public String projektEdit(@RequestParam(name = "projectId", required = false) Integer projectId, Model model) {
22         if (projectId != null) {
23             model.addAttribute("projekt", projektService.getProject(projectId).get());
24         } else {
25             model.addAttribute("projekt", new Projekt());
26         }
27         return "projektEdit";
28     }
29
30     @PostMapping("/projektEdit")
31     public String projektEditSave(@ModelAttribute @Valid Projekt projekt, BindingResult bindingResult) {
32         if (bindingResult.hasErrors()) {
33             return "projektEdit";
34         }
35         try {
36             projektService.setProjekt(projekt);
37         } catch (HttpStatusCodeException e) {
38             bindingResult.rejectValue("projekt", String.valueOf(e.getStatusCode().value()), e.getStatusCode().toString());
39             return "projektEdit";
40         }
41         return "redirect:/projektList";
42     }
43 }
  
```

9.2 StudentController

Identyczna struktura jak `ProjectController`, ale dla zasobu `Student` (endpointy `/studentList` i `/studentEdit`). Realizuje pelne zarzadzanie studentami - zadanie obowiazkowe z punktu 13 instrukcji.

```
1 package com.project.controller;
2
3 import org.springframework.stereotype.Controller;
4 import org.springframework.ui.Model;
5 import org.springframework.ui.ModelMap;
6 import org.springframework.web.servlet.mvc.annotation.AnnotationMethodMapping;
7
8 @Controller
9 public class StudentController {
10
11     private StudentService studentService;
12
13     public StudentController(StudentService studentService) {
14         this.studentService = studentService;
15     }
16
17     @GetMapping("/studentList")
18     public String studentList(Model model, Pageable pageable) {
19         model.addAttribute("studentList", studentService.getStudentList(pageable.getContent()));
20         return "studentList";
21     }
22
23     @GetMapping("/studentEdit")
24     public String studentEdit(@RequestParam(name = "studentId", required = false) Integer studentId, Model model) {
25         if (studentId != null) {
26             model.addAttribute("student", studentService.getStudent(studentId).get());
27         } else {
28             model.addAttribute("student", new Student());
29         }
30         return "studentEdit";
31     }
32
33     @PostMapping("/studentEdit")
34     public String studentEditSave(@ModelAttribute @Valid Student student, BindingResult bindingResult) {
35         if (bindingResult.hasErrors()) {
36             return "studentEdit";
37         }
38         try {
39             student = studentService.setStudent(student);
40         } catch (HttpStatusException e) {
41             bindingResult.rejectValue("id", "id.invalid", String.valueOf(e.getStatusCode().value()), e.getStatusCode().toString());
42             return "studentEdit";
43         }
44         return "redirect:/studentList";
45     }
46 }
```

10. Szablony Thymeleaf

Szablony HTML w katalogu `src/main/resources/templates/` używają dialektu Thymeleaf. Kluczowe atrybuty: `th:each` (iteracja po kolekcji), `th:text` (wyswietlanie wartosci), `th:field` (wiązanie pola formularza), `th:object` (obiekt formularza), `th:action` (adres POST), `th:if` (warunkowe renderowanie).

10.1 projektList.html

Tabela projektów z kolumnami: Id, Nazwa, Opis, Utworzony, Zmodyfikowany, Data oddania, Edit. Każdy wiersz ma linki Edytuj i Usun. Na górze linki do listy studentów i dodawania nowego projektu. Daty formatowane przez `#temporals.format`.

```

<!-- projektList.html -->
<doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<link href="/css/list-style.css" rel="stylesheet" />
<title>Lista projektów</title>
</head>
<body>
<div class="row">
<div class="col">
<a href="/studentList">Lista Studentów</a>
</div>
<div class="col">
<a href="/projektEdit">Dodaj projekt</a>
</div>
</div>
<table>
<thead>
<tr>
<th>Id</th>
<th>Nazwa</th>
<th>Opis</th>
<th>Utworzony</th>
<th>Zmodyfikowany</th>
<th>Data oddania</th>
<th>Edit</th>
</tr>
</thead>
<tbody>
<tr>
<th data-cs="7" data-kind="parent">projekt : ${projekt}</th>
<th data-kind="ghost"></th>
<th data-kind="ghost"></th>
<th data-kind="ghost"></th>
<th data-kind="ghost"></th>
<th data-kind="ghost"></th>
<th data-kind="ghost"></th>
</tr>
<tr>
<td>${projekt.id}</td>
<td>${projekt.nazwa}</td>
<td>${projekt.opis}</td>
<td>${#temporals.format(project.dataasUtworzona, 'yyyy-MM-dd HH:mm:ss')}</td>
<td>${#temporals.format(project.dataasZmodyfikacji, 'yyyy-MM-dd HH:mm:ss')}</td>
<td>${#temporals.format(project.dataasDataOddania, 'yyyy-MM-dd')}</td>
<td>
<a href="/projektEdit/${projekt.id}">Edytuj</a>
<a href="/projektEdit/${projekt.id}?delete=true">Usun</a>
</td>
</tr>
</tbody>
</table>
</body>
</html>

```

Lista projektów

[Lista Studentów](#)

[Dodaj nowy](#)

Id	Nazwa	Opis	Utworzony	Zmodyfikowany	Data oddania	Edit
2	Rozerpanie wzdłuż drzew	Projekt zaciągany	2018-06-01 21:23:28		2018-07-01	Edytuj Usun
12	Test	test	2018-06-06 16:12:28		2018-12-09	Edytuj Usun

10.2 projektEdit.html

Formularz dynamicznie zmienia nagłówki i przyciski w zależności od kontekstu: 'Utwórz projekt', 'Aktualizuj projekt' lub 'Usun projekt'. Pola: Id (readonly), Nazwa (walidacja), Opis (textarea), Data oddania (format RRRR-MM-DD), Utworzony i Zmodyfikowany (readonly). Błędy walidacji wyświetlane obok pól.

```

<!-->
<doctype html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<link th:href="@{/css/edit-style.css}" rel="stylesheet" />
<title>Edycja projektu</title>
</head>
<body>
<div class="row" th:with="isDelete=${strings.equalsIgnoreCase(param.delete,'true')}">
<form action="#" th:action="@{/project/edit}" th:object="${project}" method="POST">
  th:with="akcja=${project.id} ? (${isDelete}? 'delete':'update') : 'create', opis=${project.id} ? (${isDelete}? 'usun' : 'aktualizuj') : 'utworz'";
  autocomplete="off"

  <div th:text="${akcja} - <strong>projekt</strong> - Edytuj projekt."/>

  <div class="error" th:if="${#fields.hasErrors()}">
    <div>
      <ul>
        <li th:each="err : ${#fields.errors()}" th:text="${err}">Wprowadzone dane są niepoprawne</li>
      </ul>
    </div>
  </div>

  <div class="container">
    <div class="btn-group">
      <input class="btn" type="submit" th:name="${akcja}" th:value="${opis}" />
      <input class="btn" type="submit" name="cancel" value="Anuluj" />
    </div>
    <div th:if="${project.id}">
      <label class="id">Id:</label>
      <input th:field="${project.id}" class="fld" readonly />
    </div>
    <div>
      <label for="nazwa" class="id">Nazwa:</label>
      <input th:field="${nazwa}" class="fld" size="40" />
      <span class="err" th:if="${#fields.hasErrors('nazwa')}" th:errors="${nazwa}">Error</span>
    </div>
    <div>
      <label for="opis" class="id">Opis:</label>
      <textarea class="fld" rows="3" cols="40" th:field="${opis}">{{opis}}</textare>
    </div>
    <div>
      <label for="dataOddania" class="id">Data oddania:</label>
      <input th:field="${dataOddania}" class="fld" type="text" size="10" /><!--(RRRR-MM-DD)-->
    </div>
    <div th:if="${dataOdbioruIstnieje}">
      <label for="dataOdbioruIstnieje" class="id">Data odbioru:</label>
      <input th:field="${dataOdbioruIstnieje}" class="fld" type="text" size="10" />
    </div>
    <div th:if="${dataOdbioruNieIstnieje}">
      <label for="dataOdbioruNieIstnieje" class="id">Data odbioru:</label>
      <input th:field="${dataOdbioruNieIstnieje}" class="fld" type="text" size="10" />
    </div>
    <div th:if="${dataOdbioruNieIstnieje}">
      <label for="dataOdbioruNieIstnieje" class="id">Data odbioru:</label>
      <input th:field="${dataOdbioruNieIstnieje}" class="fld" type="text" size="10" />
    </div>
  </div>
</form>
</div>
</body>
</html>

```

Aktualizuj projekt

Anuluj

Aktualizuj

Id

0

Nazwa

Procesoryzacja danych biogeny

Opis

Projekt stworzony

Data oddania

2024-01-01 09:22:10

Data odbioru

2024-01-01 21:23:10

10.3 studentList.html

Tabela studentów z kolumnami: Id, Imie, Nazwisko, Nr indeksu, Email, Stacjonarny, Edit. Linki Edytuj i Usun w kazdym wierszu. Pole Stacjonarny wyswietla 'Tak' / 'Nie'.

```

<DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<link th:href="@{/css/list-style.css}" rel="stylesheet" />
<title>Lista studentów</title>
</head>
<body>
<div class="root">
<h2>Lista studentów</h2>
<p>
<a th:href="@{/projektlist}">Lista projektów</a>
</p>
<p>
<a th:href="@{/studentidit}">Dodaj studenta</a>
</p>
<table>
<thead>
<tr>
<th>Id</th>
<th>Imie</th>
<th>Nazwisko</th>
<th>Nr indeksu</th>
<th>Email</th>
<th>Stacjonarny</th>
<th>Idit</th>
</tr>
</thead>
<tbody>
<tr th:each="student : ${students}">
<td th:text="${student.studentId}">Id</td>
<td th:text="${student.imie}">Imie</td>
<td th:text="${student.nazwisko}">Nazwisko</td>
<td th:text="${student.nrInskau}">Nr indeksu</td>
<td th:text="${student.email}">Email</td>
<td th:text="${student.stacjonarny ? 'Tak' : 'Nie'}">Stacjonarny</td>
<td>
<a th:href="@{/studentEdit(studentId=${student.studentId})}">Edytuj</a><br>
<a th:href="@{/studentEdit(studentId=${student.studentId}, delete=true)}">Usuń</a>
</td>
</tr>
</tbody>
</table>
</div>
</body>
</html>

```

Lista studentów

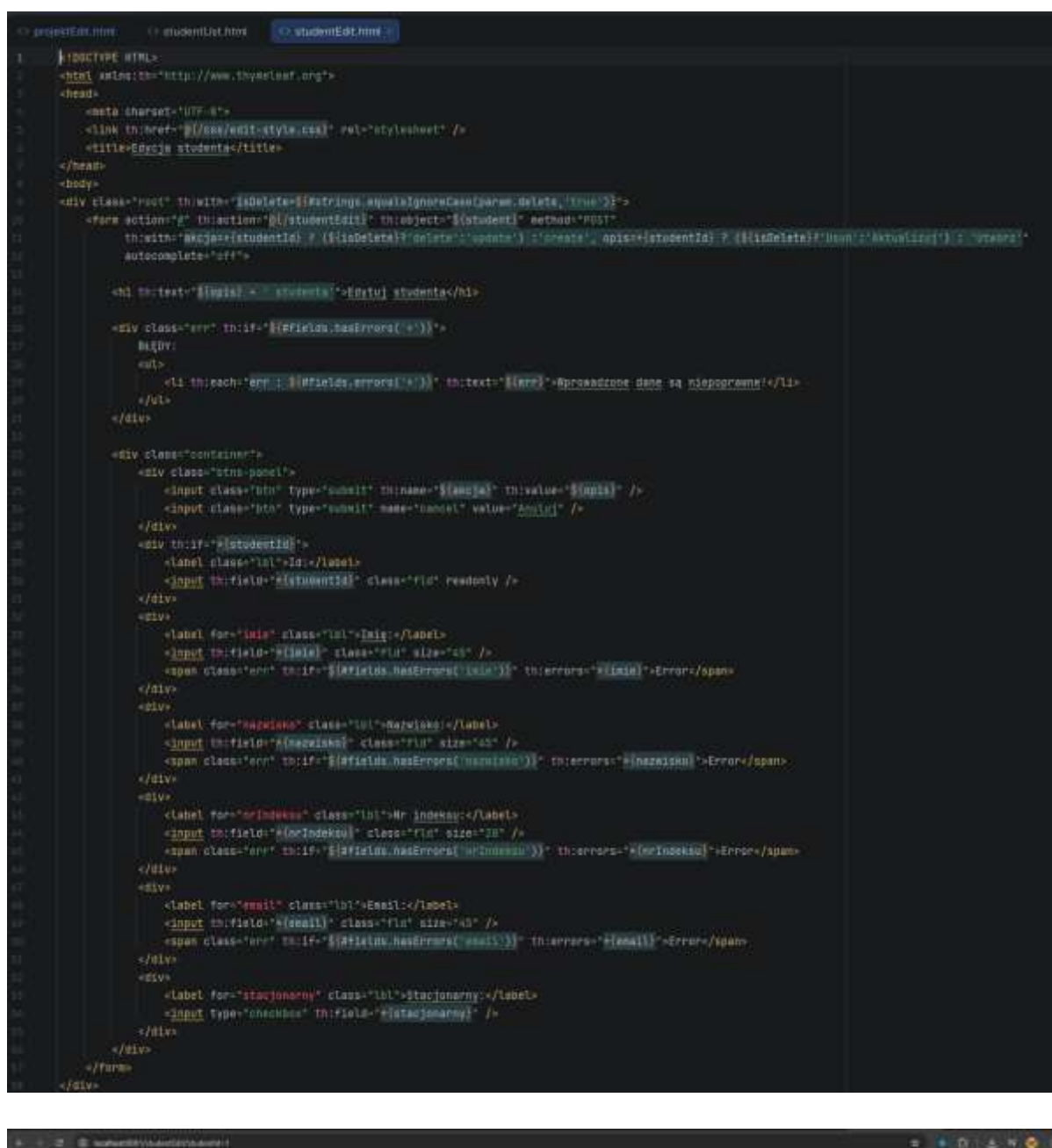
[Lista projektów](#)

[Dodaj studenta](#)

Id	Imie	Nazwisko	Nr indeksu	Email	Stacjonarny	Edit
1	Student	Imię	11111	student@ucp.pl	Tak	Edytuj Usuń

10.4 studentEdit.html

Formularz zarządzania studentem: pola studentId (readonly), Imie, Nazwisko, Nr indeksu, Email, Stacjonarny (checkbox). Dynamiczne przyciski i nagłówek jak w projektEdit.html.



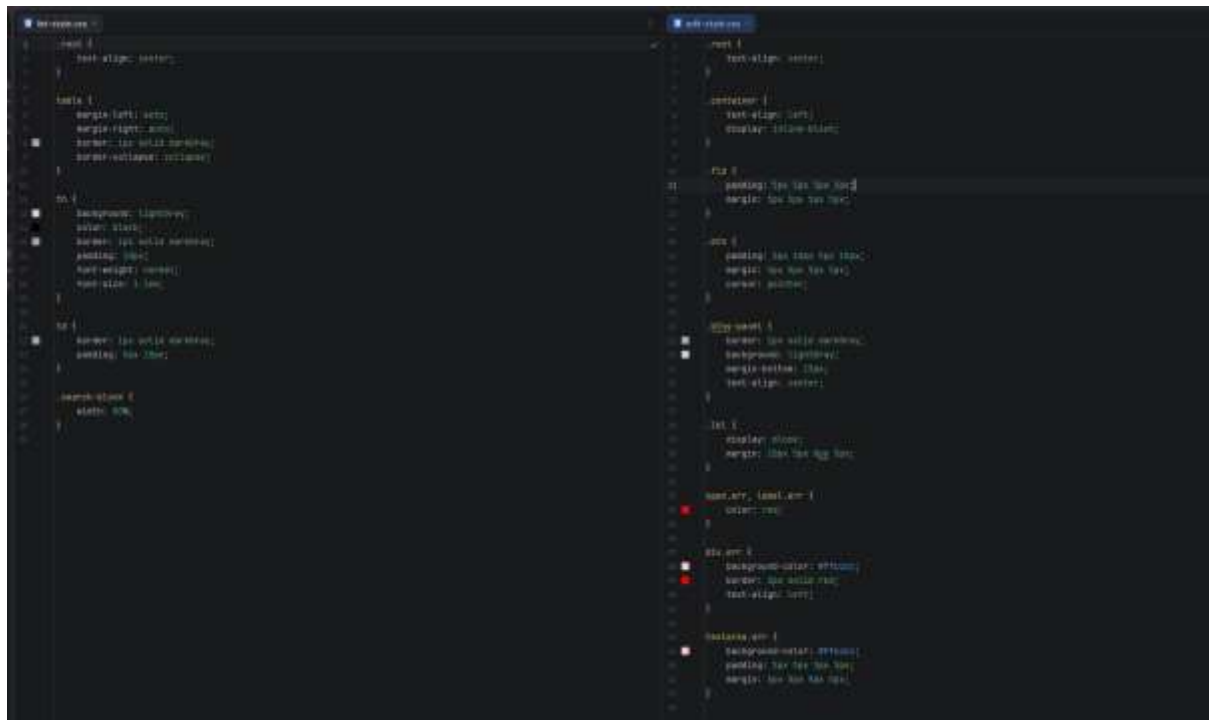
11. Style CSS

W katalogu src/main/resources/static/css/ umieszczono dwa pliki styl:

- list-style.css - style tabel list: naprzemienne kolory wierszy, wyróżnione nagłówki, ogólny wygląd tabeli.

- edit-style.css - style formularzy edycji: wyrównanie etykiet, szerokości pol input, kolory komunikatów błędów.

Szablony ładują pliki CSS przez atrybut Thymeleaf `th:href="@{/css/list-style.css}"`, co zapewnia poprawne odwołanie niezależnie od kontekstu aplikacji.



12. Konfiguracja rejestrowania (Logback)

Aplikacja używa SLF4J z Logback. Konfiguracja w `src/main/resources/logback-spring.xml` definiuje dwa appendery:

- STDOUT - wypisuje logi na standardowe wyjście (konsole IntelliJ).
- FILE - dzienny rolling appender zapisujący do pliku, historia 30 dni, max 30 MB na plik.

Logger dla pakietu `com.project` ustawiony na poziom INFO.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <configuration debug="true">
3      <property name="LOG_FILE" value="project-application" />
4      <property name="LOG_DIR" value="logs" />
5      <property name="LOG_ARCHIVE" value="${LOG_DIR}/archive" />
6
7      <!-- Send messages to System.out -->
8      <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
9          <encoder>
10              <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{30} %M(%Nline) - %msg%n</pattern>
11          </encoder>
12      </appender>
13
14      <!-- Save messages to a file -->
15      <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
16          <file>${LOG_DIR}/${LOG_FILE}.log</file>
17          <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
18              <!-- daily rollover -->
19              <fileNamePattern>${LOG_ARCHIVE}/${yyyy-MM-dd}${LOG_FILE}.log.zip</fileNamePattern>
20              <!-- Keep 30 days' worth of history capped at 30MB total size -->
21              <maxHistory>30</maxHistory>
22              <totalSizeCap>30MB</totalSizeCap>
23          </rollingPolicy>
24          <encoder>
25              <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{30} %M(%Nline) - %msg%n</pattern>
26          </encoder>
27      </appender>
28
29      <!-- For the "com.project" package and all its subpackages -->
30      <logger name="com.project" level="INFO" additivity="false">
31          <appender-ref ref="STDOUT" />
32          <appender-ref ref="FILE" />
33      </logger>
34
35      <!-- By default, the level of the root level is set to INFO -->
36      <root level="INFO">
37          <appender-ref ref="STDOUT" />
38      </root>
39  </configuration>
40
```

14. Wnioski

Laboratorium umożliwiło zapoznanie się z wzorcem architektonicznym oddzielającym aplikację webową od serwisu danych. Komunikacja wyłącznie przez REST API zapewnia niezależność warstw i ułatwia późniejsze skalowanie każdej z nich.

RestClient z konfiguracją Basic Auth w dedykowanym beanie centralnie zarządza uwierzytelnianiem, eliminując powielanie kodu. Klasy pomocnicze `RestResponsePage` i `ServiceUtil` hermetyzują szczegóły stronicowania, dzięki czemu kod serwisów pozostaje czytelny i zwięzły.

Thymeleaf okazał się efektywnym silnikiem szablonów. Atrybuty `th:object` i `th:field` znacznie upraszczają wiązanie danych formularzy z obiektami Java. Mechanizm `BindingResult` umożliwia elegancką walidację po stronie serwera bez opuszczania formularza.

Zadanie obowiązkowe (punkt 13) - zarządzanie studentami - zostało w pełni zrealizowane: zaimplementowano listę, formularz dodawania, edycji oraz usuwania studenta, zgodnie z wymaganiami instrukcji.